VEXON DEVELOPER HANDBOOK · GUIDE 09

VEXON 30-Day Learning Workbook

A daily plan to understand the code you built with AI - read real files, trace real flows, fix practice bugs, and prove it to yourself

Written for	A B.Tech student who will maintain VEXON independently after 24 October 2026
Project	VEXON / CampusConnect (repository: this folder's parent project)
Evidence	Generated from a direct inspection of the repository source code
Repository state	branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s)
Generated	16 October 2026, 21:11
Rebuild	python docs/developer-handbook/build_handbook.py

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

Contents

How this workbook works . . . 3
Day 1: Run it, map it . . . 3
Day 2: Follow one request . . . 4
Day 3: Authentication end to end . . . 4
Day 4: Roles and permissions . . . 5
Day 5: Data and MongoDB . . . 5
Day 6: Real-time with Socket.IO . . . 6
Day 7: Testing and Git . . . 7
Day 8: Debug like it is real (last day with Claude) . . . 7
Days 9 to 30: independent practice . . . 8
Self-assessment checklist . . . 9
Final practical task . . . 10
Review rubric (understanding, not memorisation) . . . 10
Source references . . . 11

How this workbook works

- **One hour or more a day**, with VS Code open on the repository and a notebook (or the blank lines in this PDF).
- Each day has: an **objective**, a short **lesson**, **reading** with real file paths, a **tracing** exercise, a **debugging** exercise, **questions**, and space for notes.
- **Days 1 to 8 (17 to 24 October 2026) are the high-leverage days.** Claude Code is still available, so use it as a **tutor and examiner, not a code writer**. Each of those days includes a "Ask Claude" box with prompts that make Claude explain or quiz you. Always try first, then ask.
- From Day 9 onwards you work with the handbook (Guides 01 to 08 and 10), the code, and the official docs.
- Never practise on production data. Use your local dev database and the seed accounts.

> **Key idea**
> The goal is not to memorise VEXON. The goal is to be able to **find out** how any part works within 20 minutes, using the code and the method in Guide 06.

Day 1: Run it, map it

Objective: start VEXON locally, then explain the folder layout from memory.

Lesson: VEXON has one backend (`server/`) and several clients (`client/`, `mobile-kotlin/`, `mobile/`, `mobile-ios/`). The server is the only authority (Guide 01).

Reading: `README.md` sections 1, 2 and 5; `package.json` (root scripts); `server/src/server.js`.

Practice:

1. Start MongoDB, then run `npm run dev`. Open `http://localhost:5173` and sign in as the seed student and the seed admin (accounts listed in `README.md`; dev only).
2. In the server terminal, find the line `[db] MongoDB connected (local) -> ...`. Which database name does it show?
3. Draw the architecture diagram from Guide 01 **without looking**, then compare.

Debugging exercise: stop MongoDB (`net stop MongoDB`, as admin) and restart the server. Read the exact error, then start MongoDB again. Which file printed it?

> **Tip**
> **Ask Claude (Day 1):** "Without changing any files, quiz me with 10 questions about VEXON's folder structure and startup sequence in server/src/server.js. Ask one at a time and tell me if I'm right."

Questions: What does `npm run seed` do to the database? Which port does the web app use, and which does the API use? Why does the web app not need CORS in development?

Day 2: Follow one request

Objective: trace the Notifications page from the click to MongoDB and back.

Lesson: a request moves through the RTK Query hook, `/api`, the middleware, the route, the controller and the model (Guide 01, "How a request moves").

Reading: `client/src/pages/Notifications.jsx`; `client/src/services/api.js` (search `getNotifications`); `server/src/routes/index.js` (search `'/notifications'`); `server/src/controllers/notificationController.js`; `server/src/models/Notification.js`.

Tracing: open DevTools, then Network, and load Notifications. Write down the request URL, status and response fields. Find the controller line that builds that response.

Debugging exercise: in the Network tab, right-click the request, choose "Copy as fetch", then change the URL to `/api/notifications/abc/read` with method PATCH and run it in the console. What status comes back, and which middleware produced it? (Expected: 422 "Invalid id" from `idParam` / `validate`, because `abc` is not an ObjectId. If you get 403 "Invalid or expired CSRF token" instead, your copied request lost its `x-csrf-token` header, and you have just watched the CSRF guard work.)

> **Tip**
> **Ask Claude (Day 2):** "I traced GET /api/notifications. Here is my written trace: <paste your notes>. Check it against the code and tell me what I got wrong. Do not edit files."

Questions: What is a cache tag? Which tags does `markNotificationRead` invalidate, and why both?

Day 3: Authentication end to end

Objective: explain sign-in, the access and refresh tokens, sessions, CSRF and the web idle rule.

Reading: `server/src/controllers/authController.js` (`login`, `refresh`, `logout`); `server/src/utils/tokens.js`; `server/src/middleware/auth.js`; `server/src/middleware/security.js`; `client/src/services/api.js` (`baseQueryWithReauth`, `refreshSession`).

Tracing: sign in with the Network tab open. Find `Set-Cookie` on the login response. Under Application, then Cookies, find `cc_refresh` and `cc_csrf`. Which one is httpOnly, and why?

Debugging exercise: sign in as the seed student, then leave the tab idle for over 2 minutes and click something. What happens, and which file decided it? (Expected: a 401 with `reason: 'idle'`, from `server/src/utils/sessionIdle.js`.) Repeat as faculty. Why is the result different?

> **Tip**
> **Ask Claude (Day 3):** "Explain refresh-token rotation and replay detection in VEXON using server/tests/auth.test.js as the example, then ask me 5 questions about it."

Questions: Why is the access token kept only in memory? What makes a request count as "mobile"? List the four layers of authorization.

Day 4: Roles and permissions

Objective: predict, before testing, whether a given role can call a given route.

Reading: `server/src/constants.js` (top 120 lines); `server/src/middleware/coeGuard.js`; `client/src/App.jsx` (`RequireAuth`, `CONFINED`); Guide 05, "Module status table".

Tracing: pick three routes: `POST /api/timetable`, `GET /api/sos`, `GET /api/hostel/summary`. For each, write the allowed roles from the code, then check with two seed accounts.

Debugging exercise: as the seed student, first call `fetch('/api/sos')` from the console. You get 401, because a raw fetch carries no Bearer token (the token lives in Redux memory). Now take a real request from the Network tab, "Copy as fetch", and change its URL to `/api/sos`. You get 403 "You do not have permission ...". Which line refused it each time?

> **Tip**
> **Ask Claude (Day 4):** "Give me 8 role-vs-route scenarios from VEXON's server/src/routes/index.js and let me predict the HTTP status for each. Then show me the code line that decides each one."

Questions: Why are client route guards not security? What is special about the first-year HOD?

Day 5: Data and MongoDB

Objective: read a model, query it safely, and explain one index.

Reading: `server/src/models/User.js` (all of it); `server/src/models/AttendanceRecord.js`; Guide 04.

Practice in mongosh (dev only): `use campusconnect`, then `db.users.countDocuments()`, then `db.users.aggregate([{ $group: { _id: '$role', n: { $sum: 1 } } }])`. Do **not** print whole user documents.

Tracing: explain `unique_attendance`. What error does a duplicate insert raise, and what status does the client see? (Expected: Mongo error `E11000`, mapped to 409 by `server/src/middleware/error.js`.)

Debugging exercise: take a `mongodump` of your dev database to a folder **outside** the repository, and restore it into `campusconnect_restorecheck`. Compare the counts, then drop the check database.

> **Tip**
> **Ask Claude (Day 5):** "Quiz me on the TTL indexes in server/src/models and what data each one deletes and when."

Questions: Why does `password` use `select: false`? What does `syncIndexes()` do to an index you delete from a model?

Day 6: Real-time with Socket.IO

Objective: explain how live updates reach the screen.

Reading: `server/src/config/socket.js`; `server/src/utils/notify.js`; `server/src/middleware/realtime.js`; `client/src/services/socket.js`; `client/src/components/layout/AppLayout.jsx` (`LIVE_TAGS`).

Tracing: open two browsers (or a normal and a private window), signed in as two seed users. Post an announcement as admin, and watch the other window update. In DevTools, Network, WS, find the `announcement:changed` frame.

Debugging exercise: stop the server while both are open, then start it again. What does the UI show while it is disconnected, and what refreshes on reconnect? Which lines in `AppLayout.jsx` do that?

> **Tip**
> **Ask Claude (Day 6):** "Explain why VEXON's sockets send only 'something changed' signals and the client re-reads over REST. What security problem does that avoid? Then quiz me."

Questions: Which rooms does every socket join? Which events need a database check before joining a room?

Day 7: Testing and Git

Objective: run every suite, read the results, and write one small test.

Reading: Guide 07; `server/tests/helpers.js`; `server/tests/sos.test.js`.

Practice: run `npm test --prefix client`, then `cd server && node --test tests/sos.test.js`, then `cd mobile-kotlin && gradlew.bat testDebugUnitTest`. Write down each result.

Write a test: on a new branch, add a test to `server/tests/sos.test.js` asserting that `POST /api/sos` with `{ "type": "party" }` returns 422. Watch it pass. Then deliberately break the rule (temporarily change `SOS_TYPES`) and watch it fail. Restore it.

Git: run `git status`. How many files are uncommitted? Make a plan for committing the existing work in logical groups (see the warning in Guide 07).

> **Tip**
> **Ask Claude (Day 7):** "Review the test I wrote in server/tests/sos.test.js. Tell me if it tests the right thing and how it could be stronger. Do not rewrite it for me."

Questions: Why does the test helper refuse non-`_test` databases? What is the difference between `git restore` and `git revert`?

Day 8: Debug like it is real (last day with Claude)

Objective: run the full Guide 06 loop on a practice bug, unaided, then use Claude only to review your work.

Setup (a practice bug on a branch): `git switch -c practice/bug`. In `client/src/services/api.js`, remove `'Notification'` from `invalidatesTags` of `markNotificationRead`.

Your task: as the student, notice that marking a notification as read does not update the list immediately. Using **only** DevTools and the code, explain why: the request succeeds, but the cache is not invalidated. Fix it, run the client tests, and write an incident report with the template in Guide 06.

Clean up: `git switch -` and `git branch -D practice/bug`.

> **Tip**
> **Ask Claude (Day 8):** "Here is my incident report for a practice bug: <paste>. Grade it against the template in docs/developer-handbook/src/06_Debugging_and_Error_Recovery_Guide.md. What evidence is missing?" Also ask: "Which three parts of VEXON would be riskiest for me to change alone, and what should I read first for each?" Save the answers in your notes.

Questions: What were the six steps of the debugging loop? Which evidence proved the cause?

Days 9 to 30: independent practice

From here on, use the handbook, the code and official documentation (React, Express, Mongoose, Socket.IO, Android). One module or skill per day.

Day	Objective	Read	Exercise (expected result)
9	JavaScript refresher	Guide 02, JavaScript section; `client/src/utils/format.js`	Rewrite `errMsg` in your own words on paper; explain each `?.`
10	React state and effects	`client/src/pages/auth/Login.jsx`	Explain the lockout countdown and its cleanup
11	Forms	`client/src/pages/complaints/ComplaintFormModal.jsx`	Explain why sub-category resets when the category changes
12	Express pipeline	`server/src/app.js`	List the middleware order from memory (14 steps)
13	Validation and errors	`server/src/middleware/validate.js`, `error.js`	Trigger a 422, a 404 and a 409 locally; match each to its code
14	Complaints module	Guide 05; `complaintController.js`	Raise a complaint as a student; follow it as the authority
15	Attendance, part 1	`attendanceController.markAttendance`	Explain every guard before the first write
16	Attendance, part 2	`server/src/services/facultyPresence.js`	Mark attendance as faculty; see the auto-present mark
17	Timetable	`timetableController.js`, `services/timetablePeriods.js`	Explain "breaks are not periods" with an example day
18	Gate pass, part 1	Gate-pass section of Guide 05	Walk a pass from request to approval with seed users (dev, SMS off)
19	Gate pass, part 2	`gatePassController.js` (`recordOut`, `recordIn`)	Explain the optimistic concurrency message "just reviewed by someone else"
20	Chat	`chatController.js`, `config/socket.js`	Explain how a private chat is authorised (`chatScope.js`)
21	Bus tracking	`services/transport/ingest.js`	List every reason a GPS fix is rejected
22	Study Assistant	`services/ai/studyAssistantService.js`	Explain how sources are chosen and why secrets are redacted
23	Kotlin structure	`mobile-kotlin/README.md`, `core/Http.kt`	Map 5 Kotlin files to their web equivalents
24	Kotlin build	Guide 08	Build a debug APK pointed at your PC (`-Pvexon.apiUrl=http://10.0.2.2:5000`)
25	Security review	Guide 07 review list; `middleware/security.js`	Explain the double-submit CSRF check in 5 sentences
26	Write a test	Guide 07	Add the club join/approve test; make it pass

Day	Objective	Read	Exercise (expected result)
27	Deployment	Guide 08; `render.yaml`	List every dashboard-only variable and what breaks without each
28	Backup drill	Guide 04	Dump, restore, compare, drop; time how long it takes
29	Full rehearsal	Guide 06	Ask a friend to break one thing on a branch; find it unaided
30	Final task	This workbook	Complete the final practical task below

Self-assessment checklist

Tick each item only if you can do it **without help, within 20 minutes**:

- [ ] Start VEXON locally from a fresh clone and a fresh `.env`.
- [ ] Explain the purpose of every top-level folder.
- [ ] Trace any button to its route, controller and model.
- [ ] Predict the status code a role gets on a route, and show the deciding line.
- [ ] Read a server stack trace and open the right file and line.
- [ ] Tell a frontend bug from a backend, database, network or configuration problem using DevTools.
- [ ] Write and run a server integration test using `helpers.js`.
- [ ] Review a diff for removed security checks and changed API contracts.
- [ ] Take and restore a MongoDB backup.
- [ ] Build the Kotlin debug APK against a local server.
- [ ] Explain the gate-pass approval chain and the parent OTP rules.
- [ ] Explain why bus positions and the viewer's own location are kept separate.

Final practical task

Choose **one** feature: Complaints, SOS, Gate Pass or Attendance marking. On paper or in a Markdown file, explain it **from the UI to the database and back**:

1. The screen and the component file(s).
2. The RTK Query hook and its URL, method and cache tags.
3. The route line, with every middleware in order and what each checks.
4. The controller, step by step, including every `ApiError` it can throw.
5. The model(s) touched, with the important fields, indexes and validation.
6. The notifications and socket events sent, and who receives them.
7. How the screen updates for the user, and for other users.
8. Which tests cover it, and one test you would add.
9. One realistic failure, how you would recognise it, and how you would fix it.

Review rubric (understanding, not memorisation)

Criterion	1: Beginning	2: Developing	3: Proficient	4: Independent
Accuracy	Several wrong files or claims	Mostly right, with vague parts	Correct files and functions	Correct, and notes edge cases
Evidence	No file paths	Some paths	Paths for every step	Paths plus line-level detail and tests
Security reasoning	Not mentioned	Mentions "auth" generally	Names `authorize`, scope and `pick`	Explains why each check exists and what would break without it
Data understanding	Unclear what is stored	Names the model	Fields, validation, indexes	Explains consistency (unique indexes, transactions)
Debugging	Cannot describe a failure	Generic failure	Realistic failure and evidence	Failure, evidence, fix and regression test
Communication	Hard to follow	Understandable	Clear and ordered	Clear enough for a new teammate to act on

A score of 3 or higher in every row means you can maintain that feature independently.

Source references

All Guides 01 to 08 and 10 · `README.md` · `server/src/` · `client/src/` · `server/tests/` · `client/tests/` · `mobile-kotlin/` · `render.yaml`